



NAME : HAMDAN ALI

WEEK 8 : Day 37

DAY 37

COMMAND USED IN DAY 37

=====

DAY 37 — COMMANDS USED (PowerShell / VS Code Terminal)

TaskFlow Capstone — Frontend UI And Backend Logic

=====

NOTE: To actually get Day 37's UI on screen, the login/register controllers and views from the Day 38 folder were also copied in at this stage (Day 37's routes/views reference "login"/"logout" named routes that only exist once those exist) — those specific commands are marked [FROM DAY38 FOLDER] below so the source is clear.

=====

PART 1 — Copying in Models

Copy-Item "..\Week8\Day37\app\Models*.php" "app\Models\" -Force

dir app\Models



PART 2 — Copying in Controllers

```
Copy-Item "..\Week8\Day37\app\Http\Controllers\*.php" "app\Http\Controllers\" -Force
```

```
dir app\Http\Controllers
```

PART 3 — Copying in Blade views

```
Copy-Item "..\Week8\Day37\resources\views\layouts" "resources\views\" -Recurse -Force
```

```
Copy-Item "..\Week8\Day37\resources\views\projects" "resources\views\" -Recurse -Force
```

```
Copy-Item "..\Week8\Day37\resources\views\tasks" "resources\views\" -Recurse -Force
```

```
Copy-Item "..\Week8\Day37\resources\views\dashboard.blade.php" "resources\views\" -Force
```

```
dir resources\views
```

```
dir resources\views\projects
```

PART 4 — Copying in Auth controllers And views [FROM DAY38 FOLDER]

(needed because Day 37's Blade layout links to /login, /register,
/logout, which don't exist without these)

```
Copy-Item "..\Week8\Day38\app\Http\Controllers\Auth" "app\Http\Controllers\" -Recurse -Force
```



```
Copy-Item "..\Week8\Day38\resources\views\auth" "resources\views\" -Recurse -Force
```

```
dir app\Http\Controllers\Auth
```

```
dir resources\views\auth
```

PART 5 — Writing routes/web.php (merged Laravel defaults + our
Day 37 routes + Day 38 auth routes)

```
# Checked what Laravel's default routes/web.php contained first  
type routes\web.php
```

```
# Overwrote it with the full merged route file
```

```
@"
```

```
<?php
```

```
use App\Http\Controllers\Auth\AuthenticatedSessionController;
```

```
use App\Http\Controllers\Auth\RegisteredUserController;
```

```
use App\Http\Controllers\CommentController;
```

```
use App\Http\Controllers\DashboardController;
```

```
use App\Http\Controllers\ProjectController;
```

```
use App\Http\Controllers\TaskController;
```

```
use Illuminate\Support\Facades\Route;
```

```
Route::middleware('guest')->group(function () {
```

```
    Route::get('/register', [RegisteredUserController::class, 'create'])->name('register');
```

```
    Route::post('/register', [RegisteredUserController::class, 'store']);
```



```
Route::get('/login', [AuthenticatedSessionController::class, 'create'])->name('login');
Route::post('/login', [AuthenticatedSessionController::class, 'store']);
});

Route::middleware('auth')->group(function () {

    Route::post('/logout', [AuthenticatedSessionController::class, 'destroy'])->name('logout');

    Route::get('/dashboard', [DashboardController::class, 'index'])->name('dashboard');

    Route::resource('projects', ProjectController::class);

    Route::post('/projects/{project}/tasks', [TaskController::class, 'store'])
        ->name('projects.tasks.store');

    Route::get('/tasks/{task}', [TaskController::class, 'show'])->name('tasks.show');
    Route::get('/tasks/{task}/edit', [TaskController::class, 'edit'])->name('tasks.edit');
    Route::put('/tasks/{task}', [TaskController::class, 'update'])->name('tasks.update');
    Route::delete('/tasks/{task}', [TaskController::class, 'destroy'])->name('tasks.destroy');

    Route::post('/tasks/{task}/comments', [CommentController::class, 'store'])
        ->name('tasks.comments.store');
});

Route::redirect('/', '/dashboard');

"@ | Set-Content -Encoding utf8 routes\web.php

# Verified file content

type routes\web.php
```



Confirmed every route registered correctly

php artisan route:list

PART 6 — First run attempt (php artisan serve)

php artisan serve

-> Visited <http://127.0.0.1:8000> in browser

-> Hit: ViteManifestNotFoundException (manifest.json missing)

PART 7 — Installing frontend build tooling (Vite/Tailwind) to fix
the missing manifest.json error

node -v

npm -v

First install attempt failed with ERESOLVE peer-dependency conflict

npm install

node_modules didn't exist yet, so nothing to clean here, but the

commands that WOULD be used to reset a broken install are:

Remove-Item -Recurse -Force node_modules

Remove-Item -Force package-lock.json

npm cache clean --force



```
# Fallback install that actually worked
```

```
npm install --legacy-peer-deps
```

```
# Built the production frontend assets (creates public/build/manifest.json)
```

```
npm run build
```

```
# Refreshed browser at:
```

```
# http://127.0.0.1:8000/login
```

PART 8 — Installing Sanctum (User model uses HasApiTokens trait, which isn't available until this is installed) [Day 38 dependency, needed here because Day 37's User model already references it]

```
composer require laravel/sanctum
```

```
php artisan vendor:publish --provider="Laravel\Sanctum\SanctumServiceProvider"
```

```
php artisan migrate
```

```
php artisan migrate:status
```

PART 9 — Debugging: registration failing with "table users has no column named role" (caused by vendor:publish resetting the role migration file back to an empty stub)



```
php artisan migrate:status
```

```
php artisan tinker --execute="print_r(Schema::getColumnListing('users'));"
```

```
# Inspected the broken migration file
```

```
type database\migrations\2026_09_04_062959_add_role_to_users_table.php
```

```
# Force-rewrote it with correct content
```

```
Set-Content -Path database\migrations\2026_09_04_062959_add_role_to_users_table.php -Encoding utf8 -Value '<?php
```

```
use Illuminate\Database\Migrations\Migration;
```

```
use Illuminate\Database\Schema\Blueprint;
```

```
use Illuminate\Support\Facades\Schema;
```

```
return new class extends Migration
```

```
{
    public function up(): void
    {
        Schema::table("users", function (Blueprint $table) {
            $table->string("role")->default("member")->after("password");
        });
    }
}
```

```
public function down(): void
```

```
{
    Schema::table("users", function (Blueprint $table) {
        $table->dropColumn("role");
    });
}
```



```
}  
};  
,
```

Verified the file content was correct this time

```
type database\migrations\2026_09_04_062959_add_role_to_users_table.php
```

Rebuilt the database clean

```
php artisan migrate:fresh
```

Confirmed "role" column genuinely exists now

```
php artisan tinker --execute="print_r(Schema::getColumnListing('users'));"
```

Went back to browser and successfully registered at:

<http://127.0.0.1:8000/register>

-> redirected to /dashboard (real working TaskFlow UI, logged in)

END OF DAY 37 COMMANDS

Result: Full UI (login, register, dashboard, projects, Kanban board, tasks, comments) live and working at <http://127.0.0.1:8000>, backed by the real Day 36 database schema, with a genuine registered user account successfully created.



Full Project View

TaskFlow

ProjectsMy TasksHamdan AliLog out

My Dashboard

2

Projects

0

Open tasks assigned to me

0

Overdue

My open tasks

Nothing on your plate right now.

TaskFlow

ProjectsMy TasksHamdan AliLog out

Your Projects

+ New Project

AI Hackathon 2026 - HindukushSoft Chitral

Active

1 tasks

TaskFlow

ProjectsMy TasksHamdan AliLog out

AI Hackathon 2026 - HindukushSoft Chitral

Edit project

TODO (0)

IN PROGRESS (1)

Login part

highUnassigned

IN REVIEW (0)

DONE (0)

Add a task

Task title

todo

medium

Add



TaskFlow

ProjectsMy TasksHamdan AliLog out

[← Back to AI Hackathon 2026 - HindukushSoft Chitral](#)

Login part

Status	Priority	Assignee	Due
In Progress	High	Unassigned	—

Comments

No comments yet.

Add a comment...

Post

TaskFlow

ProjectsMy TasksHamdan AliLog out

New Project

Name

Description

Status

Planning

Start date

mm/dd/yyyy

Create Project



EXPLANATION OF THE TASK

DAY 37

SOFTWARE ENGINEERING INTERNSHIP

Week 8 — Day 37

Frontend UI And Backend Logic Development

Turning the Day 36 plan into a working Laravel application

Project: TaskFlow — Laravel Project And Task Management System

Submitted by: Hamdan Ali

BS Software Engineering — Air University, Islamabad

Stack: Laravel 12 · PHP 8.3 · MySQL · Blade · Tailwind CSS

1. What This Task Was About

With the plan and schema locked in from Day 36, Day 37 was about actually building TaskFlow: the Eloquent models, the web controllers that hold the business logic, and the Blade views that make it usable. The brief was "develop frontend UI and backend logic," and I treated that as two halves of the same job rather than two separate tasks — every view in this folder is backed by a controller method that was written first, and every controller method exists because a route from the Day 36 plan needed an implementation.

Authentication and the JSON API are deliberately not in this folder — those are Day 38's task. Today's build assumes a logged-in user (via Laravel's `auth` middleware) and focuses purely on getting the core product working: creating projects, adding team structure, managing tasks on a Kanban board, and commenting.

2. Eloquent Models — Turning the Schema Into Code

The four models (User, Project, Task, Comment) map directly onto the five tables designed on Day 36. Rather than leaving relationships to be figured out ad hoc in controllers, I defined every relationship on the model itself, so a controller can just call `$project->tasks` or `$user->assignedTasks` and get back exactly what it needs.

The one piece of real logic living on a model rather than a controller is `Project::hasAccess($user)`. I made that decision on purpose: "can this user see this project" is a rule that gets checked in the show, update, and destroy controller actions, in the API controllers coming Day 38, and eventually in policies too. Writing that rule once on the model — owner OR admin OR a row in the members pivot — means every one of those call sites can never drift out of sync with each other. That's a habit from working with Laravel's Eloquent conventions rather than something I improvised: keep authorization logic close to the data it's protecting, not scattered across controllers.

3. Web Controllers — The Business Logic Layer

Four controllers came out of this build: `ProjectController`, `TaskController`, `CommentController`, and `DashboardController`.

- `ProjectController` handles the full project lifecycle — index (scoped to what the user can actually see, using the `visibleTo()` query scope), create/store, show (loads the project with its members and tasks, then groups tasks by status for the Kanban board), edit/update, and destroy.

- TaskController handles task creation nested under a project, and then task show/update/destroy addressed directly by task id — the shallow-nesting pattern planned on Day 36. The update method double-checks Accept headers with wantsJson() so the same endpoint can serve a full page reload or a quick AJAX status change from the board without needing two separate routes.
- CommentController is intentionally the smallest controller in the project — a single store method — because comments have exactly one thing a user can do with them at this stage (post one). I resisted the urge to over-engineer an edit/delete flow that wasn't asked for.
- DashboardController pulls together a personal view: every task assigned to the current user across every project they belong to, sorted by due date, plus three headline stats (project count, open task count, overdue count).

Validation is done inline with Laravel's validate() helper on this pass, using the exact enum values (status, priority) defined in the Day 36 migrations, so an invalid status can't even reach the database layer — the validation rule and the database ENUM constraint say the same thing on purpose.

4. Blade Views — The Frontend

The UI is server-rendered Blade with Tailwind CSS utility classes, no frontend framework — a deliberate choice for a Laravel capstone, since it demonstrates Blade competency directly rather than hiding the templating layer behind a SPA. The view structure is:

View	What it renders
layouts/app.blade.php	Shared shell: nav bar, auth state, flash message banner, @yield('content')
dashboard.blade.php	Personal stats + "my open tasks" list
projects/index.blade.php	Grid of project cards the user can access, paginated
projects/create.blade.php	Project creation form with old() value repopulation on validation errors
projects/show.blade.php	The Kanban board — one column per task status, plus an inline "add task" form
tasks/show.blade.php	Task detail: metadata, comment thread, comment form

The Kanban board in projects/show.blade.php is the piece I'm most pleased with — it iterates the four statuses defined once on the Task model (Task::STATUSES), so adding a fifth status later (e.g. blocked) is a one-line model change that the view picks up automatically instead of a template that has to be hand-edited in four different places.



5. Routing

`routes/web.php` wires every controller action up behind the auth middleware group, exactly matching the route names the Blade views already reference via Laravel's `route()` helper (`route('projects.show', $project)`, etc.), rather than hardcoded URLs — this is what makes the app resilient to a URL structure change later without breaking every link in every view.

6. Deliverables in This Folder

File / Folder	Purpose
<code>app/Models/*.php</code>	User, Project, Task, Comment Eloquent models with relationships
<code>app/Http/Controllers/*.php</code>	Project, Task, Comment, and Dashboard web controllers
<code>resources/views/**/*.blade.php</code>	Layout + dashboard + project + task Blade views
<code>routes/web.php</code>	Authenticated web route definitions

7. Reflection

Building the Kanban view was the moment the Day 36 planning paid off in a very concrete way — because `tasksByStatus` was already the right shape (grouped by the exact enum values from the schema), the view didn't need any extra logic to reshape data, it just iterated. That's the payoff of designing the database and the API/route map before touching a view: by the time you're writing Blade, you're not fighting the data, you're just displaying it.

8. References Consulted

- Capstone development walkthrough video, used as a general reference for structuring controller and view responsibilities in a Laravel capstone build.

THANK YOU!